

Evaluating Pretrained Models for Leaf Disease Detection: A study across 11 Architectures

Sahil Saifi

University Institute of Computing, Chandigarh University

Mohali, India

sahilsaifi105073@gmail.com

Abstract: This study presents a comprehensive comparison of eleven pretrained deep learning models applied to leaf disease detection using transfer learning. The models—DenseNet121, DenseNet201, EfficientNetB0, InceptionV3, MobileNet, MobileNetV2, NasNetMobile, ResNet50, ResNet152, VGG16, and VGG19—are evaluated with a focus on classification accuracy. DenseNet121 achieved the highest accuracy at 93.83%, closely followed by DenseNet201 (93.63%) and ResNet50 (92.35%). In contrast, EfficientNetB0 has the lowest accuracy of 79.09%, indicating its limitations in complex feature recognition. The results indicate that DenseNet and ResNet architectures provide superior accuracy, particularly DenseNet121 and ResNet152, which excel in capturing intricate disease patterns. Models like MobileNet and MobileNetV2 also performed well, highlighting their suitability for mobile deployment without significant accuracy trade-offs. This comparative analysis provides valuable insights into model selection for optimal leaf disease detection, depending on the computational resources and accuracy requirements.

Keywords: leaf disease detection, transfer learning, pre-trained model, DenseNet121, DenseNet201, EfficientNetB0, InceptionV3, MobileNet, MobileNetV2, NasNetMobile, ResNet50, ResNet152, VGG16, VGG19, model comparison

1. Introduction

Leaf diseases pose a significant threat to crop productivity and food security worldwide. Since they significantly lower the productivity and quality of agricultural products, they have long been one of the biggest challenges to agricultural production [1]. Thus, one of the biggest obstacles to agricultural productivity is early disease identification [2]. Early and accurate disease identification is crucial for effective disease management, preventing large-scale yield loss, and reducing the use of harmful pesticides [3] [4]. Traditional disease detection methods, which rely on manual inspection and chemical analysis, are time-consuming, labour-intensive, and often inaccurate [5]. These procedures require highly qualified professionals with experience in the sector, which can be expensive [6]. Researchers and farmers urgently need precise, quick, autonomous, and cost-effective disease detection methods. Researchers use image processing [6] [7], machine learning [8] [9], and deep learning [10] [11] to detect diseases early on. Advances in machine learning, particularly deep learning, have brought promising solutions for disease diagnosis by automating and improving accuracy through image-based classification systems.

In recent years, convolutional neural networks (CNNs) have achieved remarkable success in visual recognition tasks, including plant disease detection [11]. Transfer learning, where pretrained models are adapted to a new task, has gained traction due to its ability to leverage large-scale datasets and improve generalization on limited agricultural image datasets [12]. Using pretrained architectures for leaf disease detection enables these models to bypass the extensive training needed from scratch, conserving time and computational resources while maintaining high accuracy. This approach is especially valuable in agricultural applications where labelled data can be scarce and domain-specific features are complex.

This study focuses on evaluating the performance of eleven well-known CNN models: DenseNet121, DenseNet201, EfficientNetB0, InceptionV3, MobileNet, MobileNetV2, NasNetMobile, ResNet50, ResNet152, VGG16, and VGG19. These models, known for their high capacity to learn spatial hierarchies. The results of this comparative analysis reveal distinct trends in model performance, with DenseNet121 and DenseNet201 achieving the highest accuracies at 93.83% and 93.63%, respectively, indicating their proficiency in feature extraction and disease differentiation. Models such as MobileNet and MobileNetV2, with slightly lower accuracies but efficient lightweight structures, demonstrate potential for mobile or edge-based deployment in real-time field conditions.

This paper provides an in-depth examination of these transfer learning models' effectiveness, aiming to guide researchers and practitioners in selecting suitable architectures for specific agricultural environments. By evaluating models across various metrics, this study highlights the trade-offs between computational efficiency and classification accuracy, providing actionable insights into the selection of models that best balance performance and resource requirements.

2. Literature Review

In *"Plant Disease Detection Using Computational Intelligence and Image Processing,"* [7] Vibhor Kumar Vishnoi et al review computational techniques for plant disease identification through leaf analysis. The study examines detection systems, highlighting their strengths and limitations, and discusses feature extraction methods for various crop diseases. This work guides researchers in selecting effective tools for disease identification and classification across diverse crop types.

Applalanaidu and Gopalakrishnan, in their paper titled *"A Review of Machine Learning Approaches in Plant Leaf Disease Detection and Classification,"* [8] analyze advancements in ML and DL for plant disease detection. Reviewing over 45 studies from 2017 to 2020, they evaluate models like SVM, KNN, Naïve Bayes, and CNNs (e.g., AlexNet, VGGNet), summarizing their effectiveness in disease classification. Their analysis, organized in structured tables, offers insights into metrics like accuracy and processing techniques, providing a foundation for developing mobile applications that enhance agricultural productivity.

Prathusha et al. explore machine learning algorithms for plant disease detection in their study *"Plant Disease Detection Using Machine Learning Algorithms."* [9] They emphasize that ML methods, particularly the K-Nearest Neighbors (KNN) algorithm, offer an economical and efficient alternative to chemical pesticides, which can impact food safety. With a dataset of 15 disease types and approximately 100 images per type, the authors evaluate KNN's efficacy in categorizing diseases, highlighting its potential to aid Indian farmers by facilitating quicker, more accurate disease diagnosis.

In their paper titled *"Comparison of Deep Learning Approaches for Plant Disease Detection,"* [10] Pradhan and Patil discuss various methods for identifying plant diseases, emphasizing that traditional detection remains complex and often requires improvements in speed and accuracy. They highlight CNN models combined with deep learning as transformative tools in disease detection, offering greater precision and efficiency in agricultural applications. Their review of recent studies illustrates CNN's potential for enhancing disease detection, ultimately supporting better yield and healthier crops through proactive measures.

In *"Plant Leaf Disease Classification Using Deep Convolutional Neural Network with Bayesian Learning,"* [11] Sachdeva et al. tackle the challenge of crop disease detection amid rising global populations and static arable land. They propose a deep Convolutional Neural Network (DCNN) model utilizing a Bayesian learning process for classifying diseases in plants, using a dataset of 20,639 images from PlantVillage that includes 15 classes of healthy and diseased leaves from potato, tomato, and bell pepper. The results indicate that their DCNN outperforms traditional classifiers, achieving an accuracy of 98.9% without signs of overfitting. The authors also suggest that integrating the unique properties of nanomaterials in preprocessing can further enhance model performance, providing an effective approach for early disease detection in crops and contributing to advancements in precision agriculture.

In the paper titled *"Automation of Crop Disease Detection through Conventional Machine Learning and Deep Transfer Learning Approaches,"* [12] Houda Orchi et al. discuss the importance of tomatoes in global agriculture and the challenges posed by diseases affecting their yield. They propose a CNN-based model for detecting and classifying diseases, achieving an average accuracy of 91.2% across nine disease classes. The study compares various traditional machine learning methods with deep learning approaches, revealing that InceptionV3 outperforms others with an accuracy of 98.01%, while Naive Bayes shows the least performance at 60.09%. This research highlights the effectiveness of deep learning in agricultural disease management.

In the paper *"Deep learning for Image-Based Plant detection"* [13] the authors Prasanna Mohanty et al., have proposed an approach to detect disease in plants by training a convolutional neural network. The CNN model is trained to identify healthy and diseased plants of 14 species. The model achieved an accuracy of 99.35% on test set data. When using the model on images procured from trusted online sources, the model achieves an accuracy of 31.4%, while this is better than a simple model of random selection, a more diverse set of training data can aid to increase the accuracy. Also, some other variations of model or neural network training may yield higher accuracy, thus paving a path for making plant disease detection easily available to everyone.

In the literature review by S. Iniyan, R. Jebakumar, P. Mangalraj, Mayank Mohit, and Aroop Nanda titled *"Plant Disease Identification and Detection Using Support Vector Machines and Artificial Neural Networks,"* [14] the authors highlight the critical role of agriculture in growing economies like India, where agro products significantly impact the GDP. They emphasize the need for early disease detection to boost productivity. Their research specifically discusses crop disease detection using

machine learning techniques, focusing on support vector machines (SVM) and artificial neural networks (ANN). The review concludes by outlining the strengths and limitations of each approach concerning crop type input parameters.

S. Arivazhagan's study "*Detection of unhealthy regions of plant leaves and classification of plant leaf diseases using texture features*" [15] describes a four-step process for disease diagnosis. To process an RGB image, a color transformation structure is used. Green pixels are identified and removed using a certain threshold value. The image is then segmented, and texture statistics are computed to identify advantageous parts. Finally, the disease is classified using a classifier based on the retrieved attributes.

Orchi, Sadik, and Khaldoun's work, "*A General Survey on Plants Disease Detection Using Image Processing, Deep Transfer Learning, and Machine Learning Techniques*", [16] underlines the crucial need for efficient plant disease monitoring in Morocco to enhance agricultural productivity. Given agriculture's importance to the Moroccan economy, the paper explains how population expansion increases food consumption, producing pressure to enhance productivity. Water scarcity, insect infestations, and plant diseases are major challenges that have a negative impact on the economy due to agricultural losses.

In "*Few-Shot Learning Approach for Plant Disease Classification Using Images Taken in the Field*," [17] Argüeso et al. propose a Few-Shot Learning (FSL) model to address the limitations of large and labelled datasets by using small datasets effectively. Leveraging the PlantVillage dataset, they trained an Inception V3 network on a source domain of 32 classes, then adapted it to a target domain of 6 classes through Siamese networks and Triplet loss. The FSL model achieved 94% accuracy on small datasets, significantly reducing data requirements while maintaining high classification accuracy.

The study "*Automated Leaf Disease Detection in Different Crop Species through Image Features Analysis and One Class Classifiers*" [18] by X.E. Pantazi, D. Moshou, and A.A. Tamouridou presents an innovative approach to crop disease detection using Local Binary Patterns (LBPs) for feature extraction and One Class Classification for categorization. This approach uses a unique One Class Classifier to identify each plant health state, such as healthy, downy mildew, powdery mildew, and black rot, with a 95% success rate across 46 plant-condition combinations. The technology has been confirmed on vine leaves, but it shows great generality across diverse crops, showing its usefulness in real-world agricultural applications. This approach contains a novel conflict resolution algorithm that addresses cases where ambivalent data may correspond to several health problems.

Jatin Arora, Utkarsh Agrawal, and Perna Sharma's work "*Classification of Maize Leaf Diseases from Healthy Leaves Using Deep Forest*" [19] underlines the need of precise and efficient plant disease detection in supporting agriculture's role in global food security and economic stability. This work proposes a unique application of the Deep Forest method to improve the classification and recognition accuracy of maize leaf diseases. The authors suggest that this approach outperforms typical deep neural networks and machine learning models, with less reliance on hyperparameter tuning and dataset size, indicating a significant development in automated agricultural diagnostics.

Zhe Tang et al.'s work "*Grape disease image classification based on lightweight convolution neural networks and channelwise attention*," [20] offer a compact convolutional neural network (CNN) model for identifying grape diseases such as black rot, black measles, and leaf blight. This model focuses on low latency and small size, making it ideal for mobile deployments. To improve the model's performance, channelwise attention (CA) techniques are included into the ShuffleNet V1 and V2 backbones via squeeze-and-excitation (SE). Testing on an open dataset of 4,062 pictures across four classes (three diseased and one healthy), the model demonstrated a high accuracy of 99.14% while reducing model size from AlexNet's 227.5 MB to a lightweight 4.2 MB, demonstrating its promise for on-device grape disease diagnosis.

Anshul Bhatia, Anuradha Chug, and Amit Prakash Singh's study "*Application of Extreme Learning Machine in Plant Disease Prediction for Highly Imbalanced Dataset*" [21] uses an Extreme Learning Machine (ELM) algorithm to predict plant diseases using the Tomato Powdery Mildew Disease (TPMD) dataset. Given the imbalance in the dataset, resampling techniques like Importance Sampling (IMPS), Synthetic Minority Over-sampling Technique (SMOTE) [22], Random Under Sampling (RUS), and Random Over Sampling (ROS) [23] were applied for balance. ELM models were created for each balanced dataset as well as for the imbalanced one. Performance was measured by Area Under Curve (AUC) and Classification Accuracy (CA). The IMPS technique yielded the best results, with CA and AUC reaching 89.19% and 88.57%, respectively. This study highlights the benefits of resampling in improving prediction outcomes.

In their study "*ToLeD: Tomato Leaf Disease Detection Using Convolution Neural Network*," [24] Agarwal et al. present a CNN model tailored for detecting diseases in tomato leaves, a critical task for maintaining crop quality, especially given India's high tomato production. Their custom CNN architecture, with three convolutional and max-pooling layers followed by two fully connected layers, surpassed pretrained models like VGG16 and InceptionV3, achieving an average accuracy of 91.2% across nine disease categories and one healthy class. This custom approach demonstrates promising accuracy and specificity in tomato disease classification.

Gandhi et al., in their paper titled “*Plant Disease Detection Using CNNs and GANs as an Augmentative Approach*, ” [25] address crop loss challenges in regions like Maharashtra, where crop failures impact farmer livelihoods. They propose a CNN-based classification system for plant disease detection tailored for Indian crops, using Generative Adversarial Networks (GANs) to augment the limited local image dataset. This enhanced dataset, deployed through a smartphone app, offers accessible, precise disease detection for Indian farmers, aiming to mitigate crop loss and improve early intervention.

In “*Machine Learning-Based Tea Leaf Disease Detection: A Comprehensive Review*, ” [26] Ahmed et al. systematically reviews the application of machine learning techniques for diagnosing tea leaf diseases through image classification. They emphasize the importance of early detection in managing crop yield and quality in the tea industry. The paper evaluates various Vision Transformer models, such as Inception Convolutional Vision Transformer (ICVT), GreenViT, and others, alongside traditional models like DenseNet, ResNet-50V2, and YOLO variants. By assessing the strengths and limitations of these methodologies, the authors highlight significant advancements in machine learning for tea leaf disease detection and offer insights for future research directions in this area.

In “*A Precise Image-Based Tomato Leaf Disease Detection Approach Using PLPNet*, ” [27] Tang et al. (2023) tackles the challenges of detecting tomato leaf diseases, which can significantly impact crop yield. They introduce PLPNet, featuring a perceptual adaptive convolution module to effectively capture disease characteristics and a location reinforcement attention mechanism to reduce soil background interference. The proximity feature aggregation network enhances feature consistency and addresses interclass similarities among diseases. Their experimental results show PLPNet achieving a mean average precision (mAP) of 94.5% and an average recall of 54.4%, demonstrating superior accuracy compared to existing detection models, thereby improving modern tomato cultivation management.

Mehta et al.'s work “*Advancing Agricultural Practices: Federated Learning-based CNN for Mango Leaf Disease Detection*, ” [28] introduce a CNN model utilizing federated learning to enhance the identification of mango leaf diseases, which significantly threaten crop yields and farmer livelihoods. The study evaluates the model across four clients and five disease categories, reporting precision values from 93.33% to 96.01%, recall from 90.59% to 97.45%, and F1-scores between 92.64% and 96.10%. With an accuracy range of 97% to 98%, the model demonstrates consistent performance, supported by robust macro, weighted, and micro averages. This federated approach not only improves disease detection but also preserves data privacy, promoting collaborative learning among clients. The research contributes to enhanced agricultural practices, fostering better crop management and food security.

3. Methodology

We conducted a thorough comparative research of multiple Deep Learning models for the case study of 38 different leaf classes on Python utilizing many libraries such as Numpy, Pandas, Scikit-learn, Keras, and TensorFlow.

3.1 Experimental Setup

The study used a Google Compute Engine instance called Google Collaboratory and a local Asus Vivobook workstation with 8 GB RAM. The Collaboratory Notebook [29] is driven by Jupyter, which functions similarly to Google Docs. The notebooks are pre-configured with top machine learning libraries, including Keras, TensorFlow, and Matplotlib. The Google Collaboratory offers fast TPUs and runs on Ubuntu 17.10 64-bit. This system consists of an Intel Xeon CPU with 13 GB of RAM and an NVIDIA Tesla K80 processor with 12 GB of RAM and 2496 CUDA.

3.2 Proposed Methodology

Figure 1 shows the system for identifying leaf diseases over multiple phases. We pre-processed raw photos acquired from the source [30]. Data augmentation was already done. The pre-processed data was divided into two sets: training and validation. In Phase 2, the training data was used to train the selected models. We evaluated our models' performance using the precision, accuracy, recall, and F1-score.

3.3 Data Collection

Our data is derived from the New Plant Diseases Dataset [29]. There are 87,867 photos of 14 different plant leaves, organized into 38 disease classes: healthy leaf and diseased leaf. The dataset is recreated using offline augmentation from the original dataset. The original dataset can be found on this GitHub repository [30].

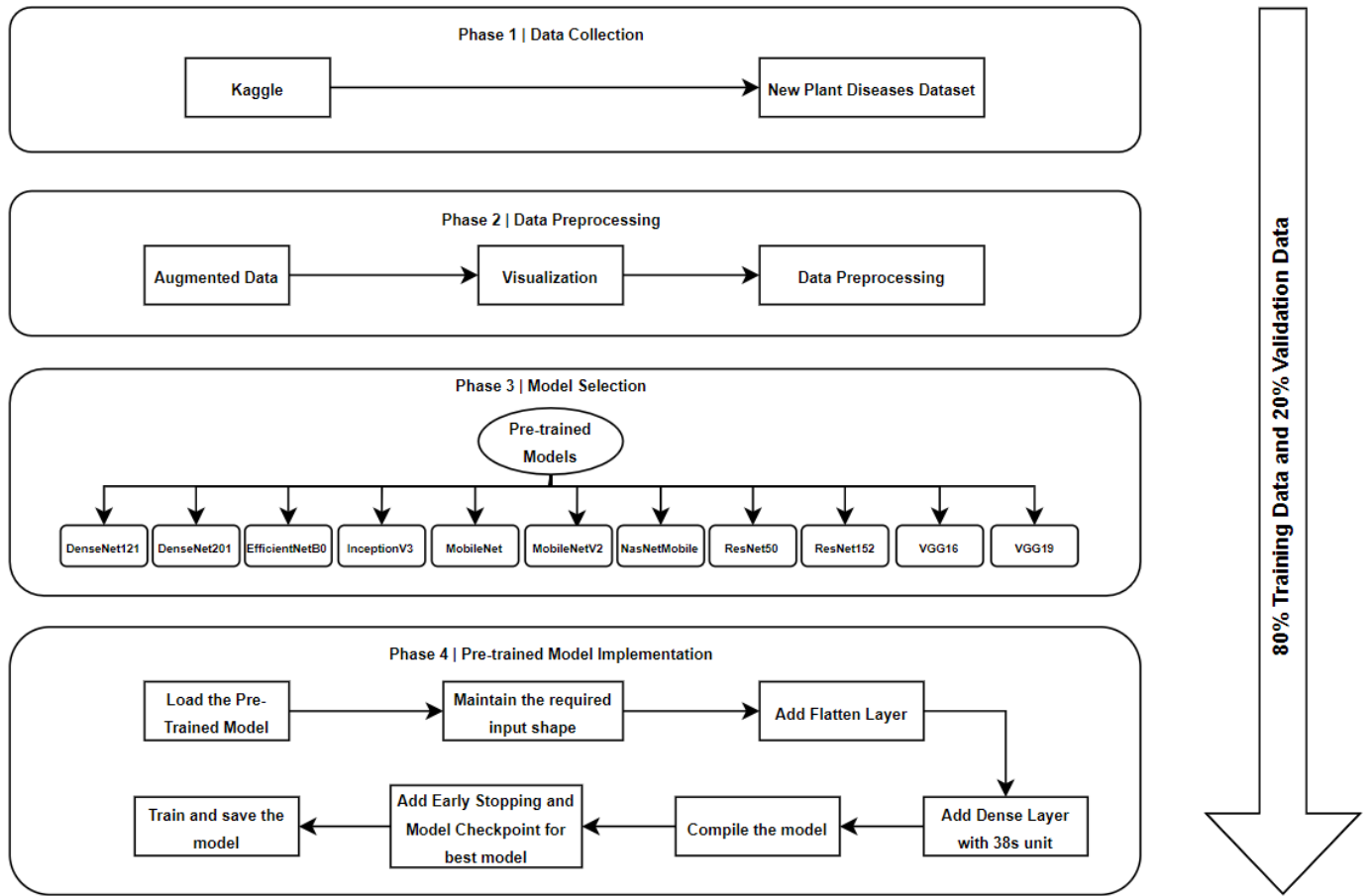


Figure 1. A high-level overview of the process

Table 1. The detailed dataset was extracted from the New Plant Disease Dataset.

Crop Leaf	Diseases	Number of Images
Apple	Apple scab	2520
	Black rot	2484
	Ceder apple rust	2200
	Healthy	2510
Blueberry	Healthy	2270
Cherry	Healthy	2282
	Powdery mildew	2104
Maize	Leaf spot	2052
	Common rust	2384
	Healthy	2324
	Leaf blight	2385
Grape	Black rot	2360
	Black measles	2400
	Healthy	2115
	Leaf blight	2152
Orange	Citrus greening	2513
Peach	Bacterial spot	2297
	Healthy	2160
Bell Pepper	Bacterial spot	2391
	Healthy	2485
Potato	Early blight	2424
	Healthy	2280
	Late blight	2424
Raspberry	Healthy	2226
Soybean	Healthy	2527
Squash	Powdery mildew	2170

Table 1. *Contt.*

Crop Leaf	Diseases	Number of Images
Strawberry	Healthy	2280
	Leaf scorch	2218
Tomato	Bacterial spot	2127
	Early blight	2400
	Healthy	2407
	Late blight	2314
	Leaf mold	2352
	Leaf spot	2181
	Spider mite	2176
	Target spot	2284
	Mosaic virus	2238
	Yellow leaf curl virus	2451

3.4 Data Preprocessing

The data is already augmented with the and derived form the original dataset [31]. Crop leaf images are 256×256 pixels in RGB color. A few randomly selected leaf images from the dataset are shown to demonstrate the image format after eliminating potential bias, color change, backdrop suppression, and contrast tuning are necessary, as shown in Figure 2. This collection of photos features the following species: apple, blueberry, cherry, maize, grape, orange, peach, bell pepper, potato, raspberry, soybean, squash, strawberry, and tomato. These crop leaves are susceptible to viral, fungal, and bacterial infections. Table 1 contains a detailed presentation of the dataset. It is worth noting that the image distribution is not uniform, resulting in a data imbalance. Figure 3 shows a bar chart depicting the distribution of photos per class.

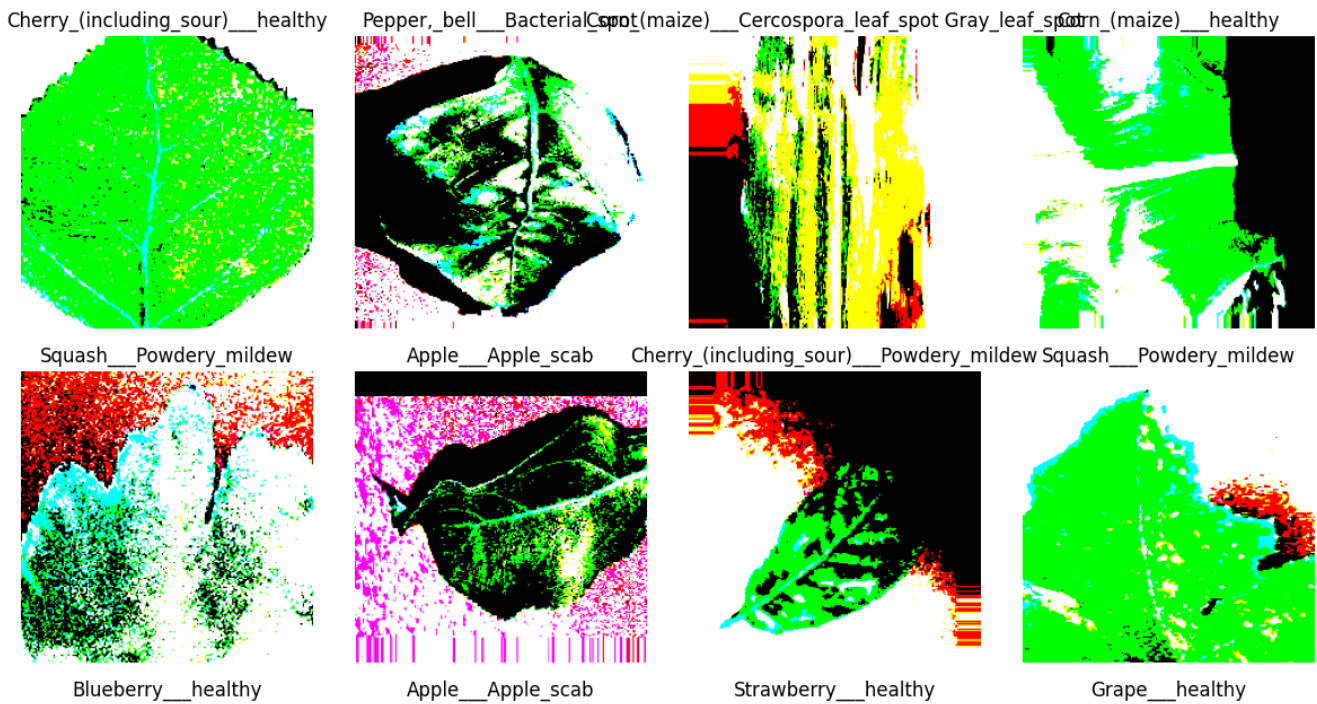


Figure 2. An overview of various crop leaf photos taken randomly from the New Plant Disease Dataset after data preprocessing.

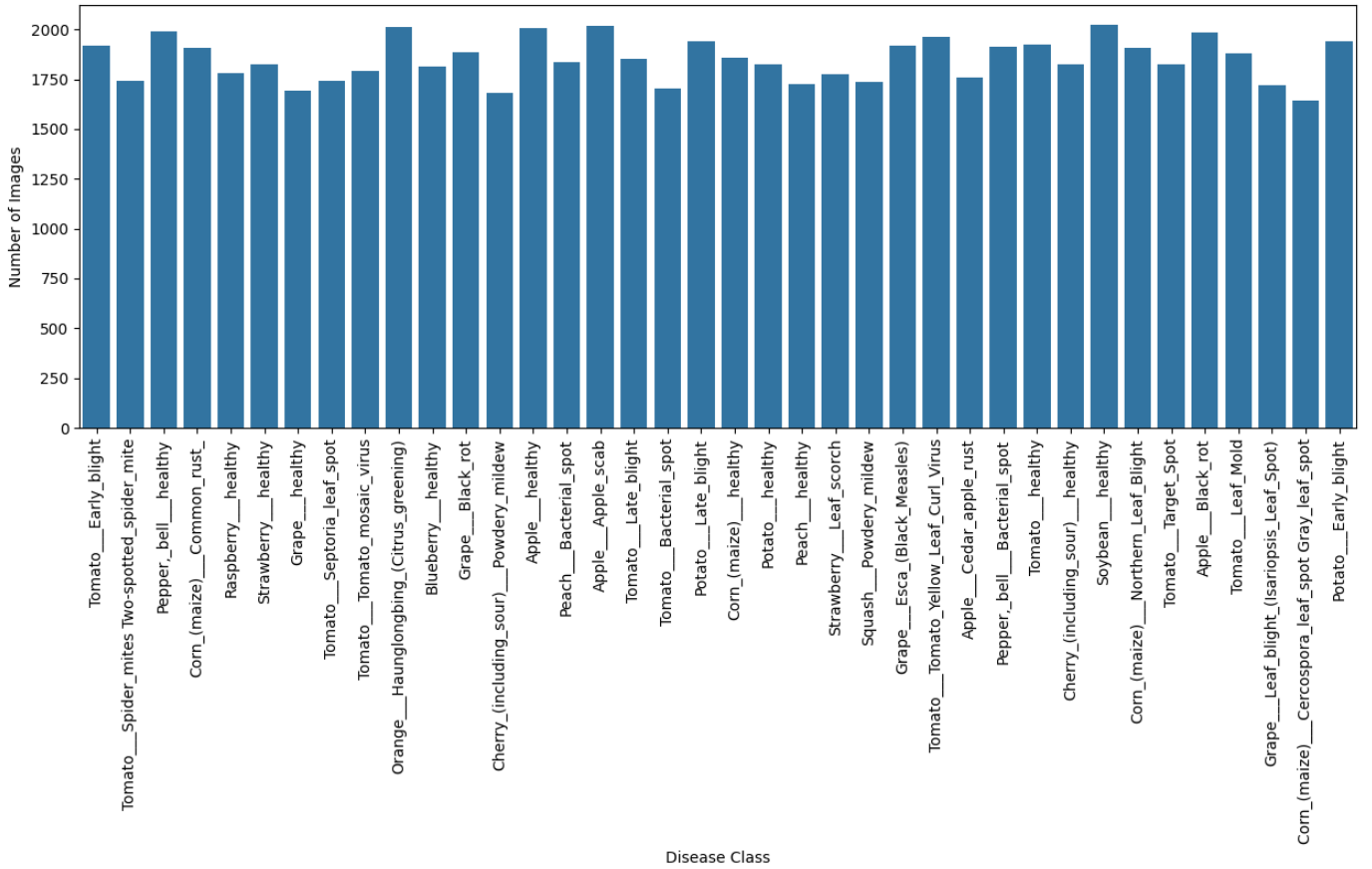


Figure 3. Distribution of photos per class

3.5 Model Specification

3.5.1 DenseNet121

DenseNet121 is a type of convolutional neural network (CNN) architecture introduced as part of the DenseNet family by Huang et al. in 2017 [32]. DenseNet, short for "Densely Connected Convolutional Networks," is known for its unique structure, where each layer is directly connected to every other layer in a dense block. This connectivity helps in feature reuse and minimizes the number of parameters, making DenseNet more efficient compared to traditional architectures.

- **Dense Blocks:** DenseNet121 consists of four dense blocks, with each block containing multiple convolutional layers. Inside these blocks, each layer receives feature maps from all previous layers, which are then concatenated to form an input for the next layer. This dense connectivity enables better gradient flow, improves feature propagation, and reduces the risk of vanishing gradients.
- **Bottleneck Layers:** Each convolutional layer within the dense blocks follows a "bottleneck" design, with a 1×1 convolution followed by a 3×3 convolution. This bottleneck reduces the number of feature maps, optimizing computation.
- **Transition Layers:** Between dense blocks, DenseNet121 includes transition layers. Each transition layer consists of a 1×1 convolution followed by 2×2 average pooling, which reduces the spatial dimensions and controls the model's complexity.
- **Output Layer:** DenseNet121 ends with a global average pooling layer and a fully connected (dense) layer for classification.
- **Parameters:** DenseNet121 has 121 layers, hence the name, and is more computationally efficient than other deep architectures, due to its dense connections and use of fewer filters per layer.

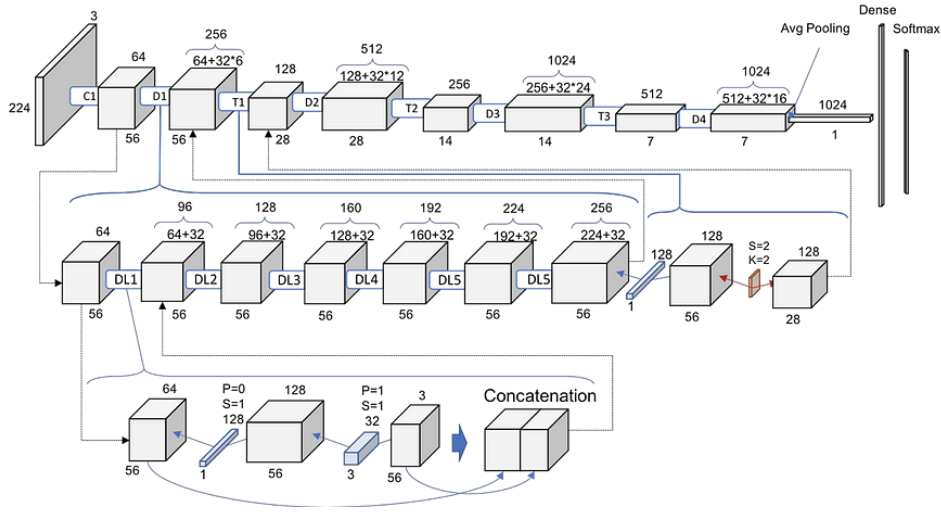


Figure 4. Architecture of DenseNet121 [33]

3.5.2 DenseNet201

DenseNet201 is part of the DenseNet family of architectures, which are designed with densely connected convolutional networks to enhance feature propagation and reduce the number of parameters. As an extension of DenseNet121, DenseNet201 contains 201 layers and uses dense connections to improve gradient flow and feature reuse.

- **Dense Blocks:** DenseNet201 consists of four dense blocks, with each block containing a series of convolutional layers. In these blocks, each layer receives the output feature maps from all previous layers within the block as additional inputs. This connectivity pattern helps reduce the number of parameters and improve gradient flow, making training more efficient.
- **Bottleneck Layers:** DenseNet201 employs bottleneck layers with a 1×1 convolution followed by a 3×3 convolution. These layers reduce the number of feature maps, minimizing computational costs and enhancing model efficiency.
- **Transition Layers:** Between each dense block, transition layers are introduced to down sample feature maps. Each transition layer includes a $1 \times 11 \times 11$ convolution and a 2×2 average pooling layer. These layers reduce the spatial dimensions, effectively managing the complexity and size of the model.
- **Growth Rate:** DenseNet201 has a growth rate, which is the number of new feature maps added by each layer. This rate is kept relatively low to control model size, as each dense layer concatenates its feature maps with those from previous layers.
- **Global Average Pooling and Classification Layer:** DenseNet201 concludes with a global average pooling layer followed by a fully connected layer for classification, enabling the model to adapt to various image classification tasks.
- **Parameter Efficiency:** Compared to traditional CNNs, DenseNet201 achieves higher accuracy with fewer parameters, due to its dense connections and efficient layer design. This efficiency makes DenseNet201 suitable for deep learning applications requiring large-scale image classification with limited computational resources.

3.5.3 EfficientNetB0

EfficientNetB0 is the foundational model of the EfficientNet family, introduced by Google researchers in 2019 [34]. This architecture is known for its innovative scaling approach, which balances network depth, width, and resolution to optimize both accuracy and computational efficiency. EfficientNetB0 was developed using a compound scaling method that achieves higher accuracy with fewer parameters and lower computational costs than many other deep learning models.

- **Compound Scaling:** EfficientNetB0 introduces a compound scaling technique, where depth (number of layers), width (number of channels), and resolution (input image size) are scaled together in a balanced manner. This differs from traditional methods that increase these factors independently, often leading to resource inefficiencies. In EfficientNetB0, the model is optimized by scaling these dimensions harmoniously, providing a good balance of performance and efficiency.
- **MBConv Layers:** EfficientNetB0 uses a variant of mobile inverted bottleneck convolutions (MBConv), originally from MobileNetV2, which includes depthwise separable convolutions. MBConv layers reduce the number of parameters by splitting the standard convolution process, making the model more efficient while maintaining accuracy.
- **Squeeze-and-Excitation (SE) Blocks:** To improve the model's ability to focus on important features, EfficientNetB0 includes Squeeze-and-Excitation blocks within MBConv layers. These blocks adaptively recalibrate channel-wise

feature responses, enhancing the network's sensitivity to informative features and improving its classification performance.

- **Lightweight and High Accuracy:** Despite its small size and low computational demand, EfficientNetB0 achieves impressive accuracy on image classification benchmarks. With just 5.3 million parameters, it achieves accuracy comparable to or better than models with far more parameters, such as ResNet and VGG.
- **Transfer Learning Potential:** EfficientNetB0 is a popular choice for transfer learning in various applications, including medical imaging and agricultural disease detection, due to its efficiency and high performance on small datasets.

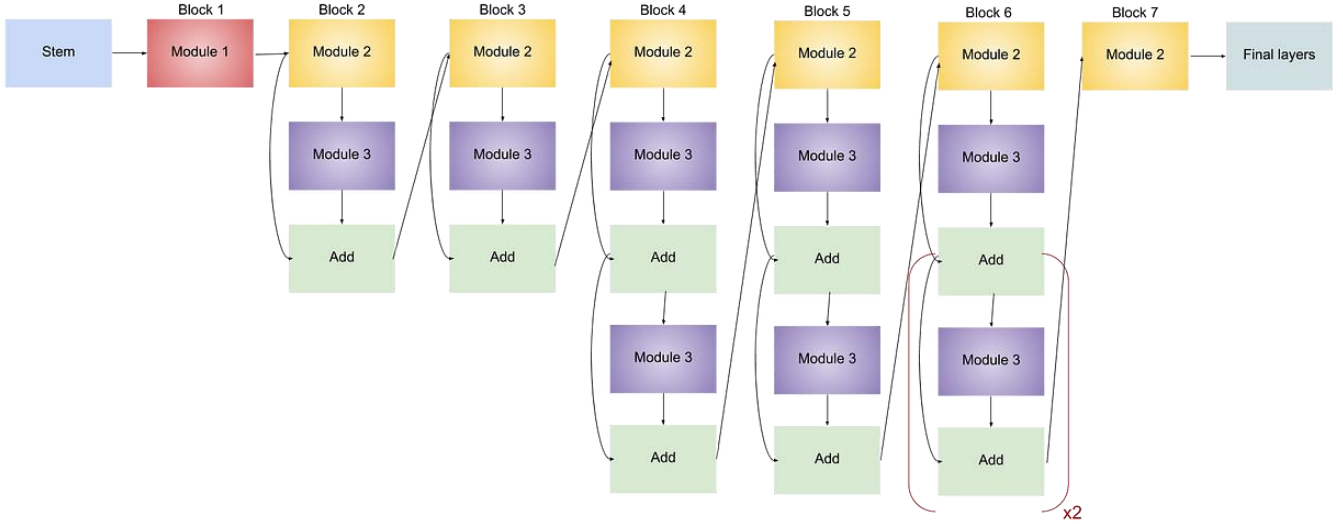


Figure 5. Architecture of EfficientNetB0 [35]

3.5.4 InceptionV3

InceptionV3 is a popular architecture for convolutional neural networks developed by Google. InceptionV3 uses multiple convolution filters of varying widths to detect spatial patterns in input images. InceptionV3's performance and efficiency make it ideal for image processing and classification jobs. The use of 1×1 convolutions enhances the architecture and improves computing efficiency for dataset training and inference [36].

InceptionV3 uses batch normalization to improve model robustness and speed up training. Additionally, this model offers optimized grid sizes and factorized convolutional layers. InceptionV3 includes auxiliary classifiers for improved performance and accuracy. [37]

InceptionV3 effectively balances complexity and efficiency by breaking down larger convolutions into smaller ones. The InceptionV3 architecture offers auxiliary classifiers, which enhance gradient signals and increase convergence during dataset training. InceptionV3 uses label smoothing to prevent overfitting. InceptionV3 has revolutionized image processing and set a new standard for future CNN architectures.

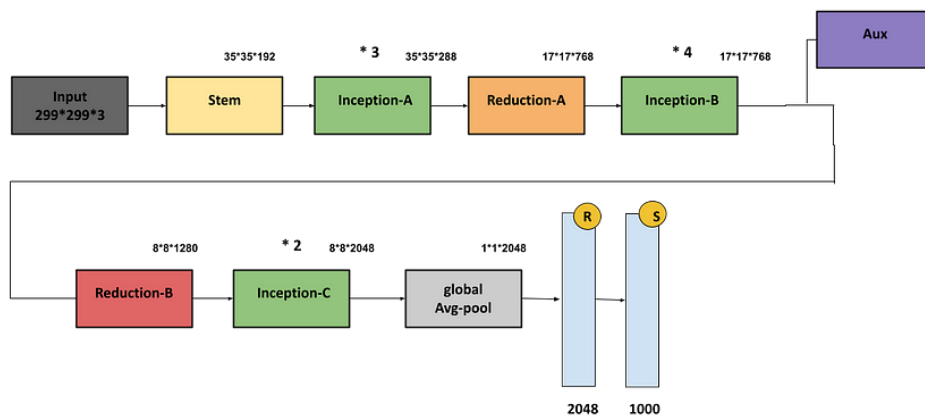


Figure 6. Architecture of InceptionV3 [38]

3.5.5 MobileNet

MobileNet is a lightweight convolutional neural network architecture designed specifically for mobile and embedded devices, focusing on efficiency and speed [39]. Its main innovation is the use of depth wise separable convolutions, which break down standard convolution operations into two separate steps:

- **Depth wise Convolution:** This applies a single filter to each input channel, drastically reducing computation by avoiding multi-channel convolutions.
- **Pointwise Convolution:** A 1×1 convolution combines the output from the depth wise convolutions, finalizing the feature map.

This two-step process significantly reduces the number of parameters, making MobileNet computationally efficient. MobileNet also includes width and resolution multipliers, which allow further control over the trade-off between model size, speed, and accuracy, making it adaptable to different device capabilities.

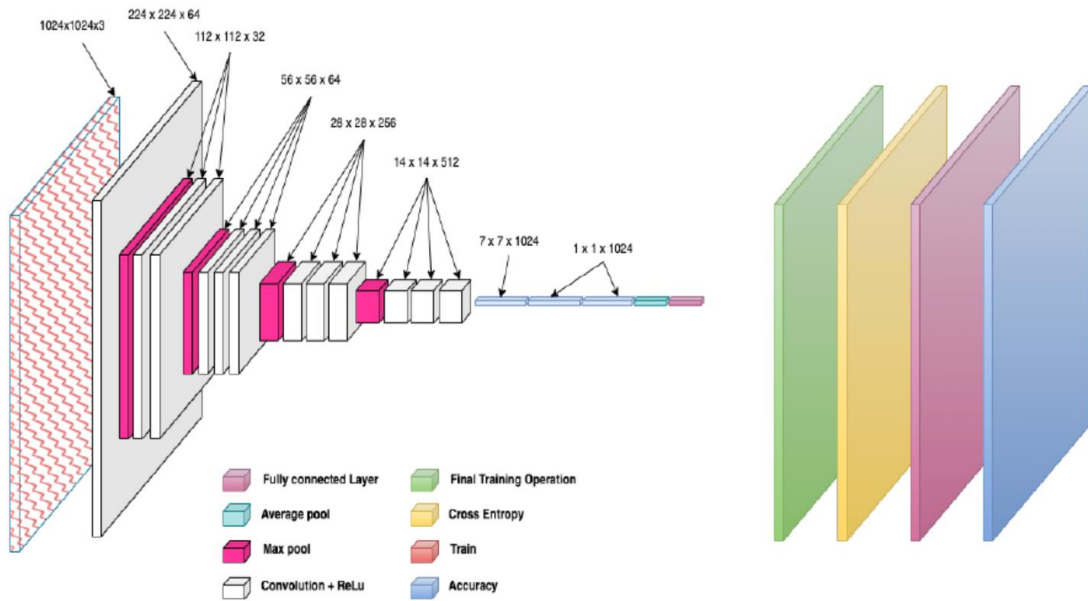


Figure 7. Architecture of MobileNet V1 [40]

3.5.6 MobileNetV2

MobileNetV2 builds upon MobileNet with further improvements aimed at achieving higher accuracy and efficiency [39]. It introduces an inverted residual structure with linear bottlenecks:

- **Inverted Residuals:** Instead of the traditional approach of shrinking and expanding dimensions within residual blocks, MobileNetV2 begins with a 1×1 convolution to expand the feature space, followed by a depth wise convolution, and then a final 1×1 convolution to reduce channels (the "bottleneck").
- **Linear Bottlenecks:** By keeping the bottleneck layers linear, MobileNetV2 minimizes information loss during transformations, ensuring that significant features are retained.

These design improvements allow MobileNetV2 to perform better with lower computational costs, making it highly efficient for mobile and low-power devices.

While MobileNet uses depth wise separable convolutions to reduce model complexity, MobileNetV2 enhances this with inverted residuals and linear bottlenecks for better feature retention and accuracy on lightweight devices.

3.5.7 NasNetMobile

The NASNet (Neural Architecture Search Network) architecture, introduced by Zoph et al. [41] [42], is a pioneering deep learning model developed to achieve high performance through automated architecture search. NASNet leverages Neural Architecture Search (NAS), a method that employs a reinforcement learning controller to iteratively generate and evaluate candidate network structures. This automated search explores a vast space of architectures to identify the optimal network configuration, balancing model accuracy with computational efficiency. The NAS controller is guided by a reward signal based on validation accuracy, continually refining its search process to maximize model performance.

The architecture of NASNet is based on a modular cell-based design. The model consists of two types of cells—Normal Cell and Reduction Cell:

- **Normal Cell:** This cell maintains the spatial dimensions of the input feature map, making it useful for deep layers without reducing resolution.
- **Reduction Cell:** This cell, on the other hand, reduces spatial dimensions, providing an efficient mechanism for down sampling. This alternation between normal and reduction cells allows NASNet to control resolution and computational complexity effectively.

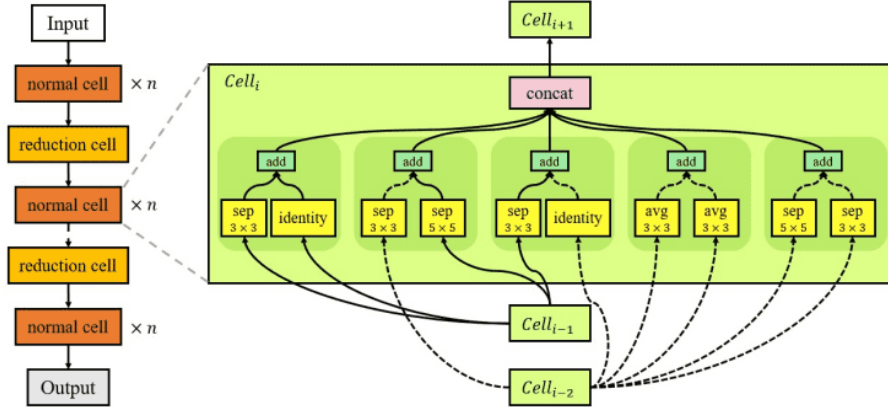


Figure 8. Architecture of NasNet

3.5.8 ResNet50

ResNet50, part of the ResNet (Residual Networks) family introduced by He et al. (2015), is a deep convolutional neural network with 50 layers. This architecture is built around residual blocks that use shortcut connections to bypass certain layers, allowing the model to learn residual functions rather than direct mappings. Each residual block consists of numerous layers (convolution, batch normalization, and ReLU) with a skip connection that adds a block's input to its output, so mitigating the vanishing gradient problem that can arise in deep networks.

ResNet50 follows a conv-block structure, where each block consists of a series of three convolutions with increasing filter sizes (1x1, 3x3, and 1x1). The initial 1x1 convolution reduces the dimensions, the 3x3 convolution extracts features, and the final 1x1 convolution restores the dimensions. This allows for efficient computation and depth. With 50 layers, ResNet50 achieves high performance on visual recognition tasks while remaining relatively lightweight compared to deeper ResNet models.

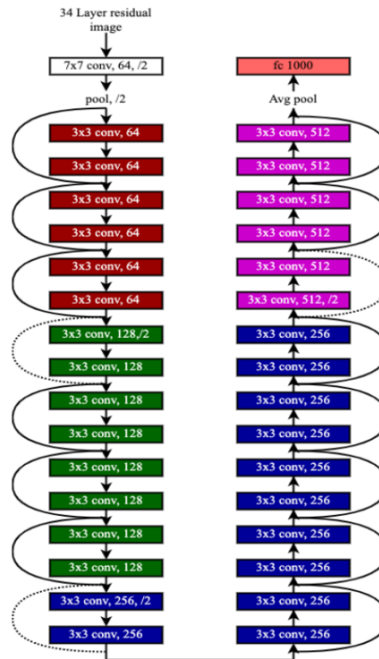


Figure 9. Architecture of ResNet50 [40]

3.5.9 ResNet152

ResNet152 builds upon the design principles of ResNet50 but with an increased depth of 152 layers. This additional depth is achieved by stacking more residual blocks, enhancing the model's capacity to learn complex features. Like ResNet50, ResNet152 utilizes the same residual connections, which prevent degradation of performance and ensure that adding more layers does not lead to a higher training error. However, due to its greater depth, ResNet152 can capture more intricate patterns, which leads to improved performance on tasks like image classification, but also increases computational and memory demands.

In terms of structure, ResNet152 is composed of similar conv-blocks as ResNet50, but it has more layers per block, allowing for finer feature extraction and higher-level feature representations.

The primary difference between ResNet50 and ResNet152 is the network depth, with ResNet50 having 50 layers and ResNet152 having 152 layers. This increased depth in ResNet152 provides better feature learning at the cost of higher computational requirements, making ResNet152 more suited to tasks needing extremely high accuracy and deeper feature extraction.

3.5.10 VGG16

VGG16 [43], introduced by the Visual Geometry Group at Oxford, is a convolutional neural network (CNN) architecture with 16 layers. Its design is characterized by uniformly sized convolutional filters of 3x3, applied consistently throughout the network. VGG16 follows a straightforward, sequential structure: blocks of convolutional layers are followed by max-pooling layers to reduce spatial dimensions while preserving essential features. After several convolutional and max-pooling layers, three fully connected layers are added before the final softmax output layer. This architecture emphasizes depth and simplicity, utilizing only 3x3 convolution filters and 2x2 max pooling, with each convolutional layer's output gradually increasing in depth (64 to 512 filters) to capture intricate patterns. VGG16 achieves high accuracy on visual recognition tasks while maintaining a manageable size and computational efficiency.

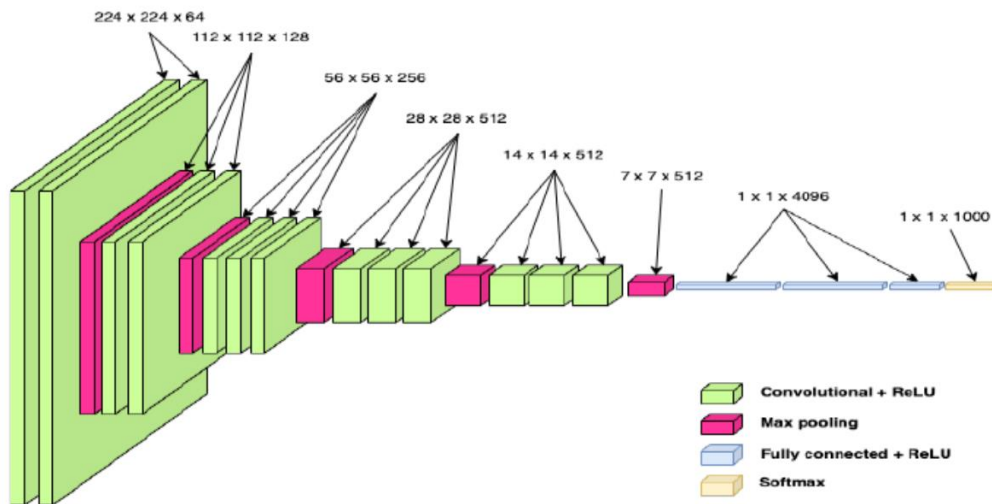


Figure 10. Architecture of VGG16 [40]

3.5.11 VGG19

VGG19 builds on the VGG16 design, increasing its depth to 19 layers by adding additional convolutional layers in some of the blocks. This added depth allows VGG19 to capture even more complex and nuanced patterns in data, potentially improving classification accuracy for more challenging visual recognition tasks. Like VGG16, VGG19 exclusively uses 3x3 convolutional filters and 2x2 max-pooling layers, ensuring uniformity across the model. However, the increased depth makes VGG19 more computationally intensive, requiring more memory and processing power than VGG16 [43].

The main difference between VGG16 and VGG19 is the number of layers, with VGG16 having 16 layers and VGG19 having 19 layers. This additional depth in VGG19 can yield better accuracy on complex datasets but at the cost of higher computational resources.

4. Implementation

4.1 Workflow

Data collecting from the source for various leaf kinds was the first step in the research. To ensure a varied depiction of leaf condition and disease manifestation, this required a significant amount of data collecting and preprocessing.

The dataset was meticulously divided into sections designated for testing and testing. The deep learning models were trained using about 80% of the data, their performance was validated using 20% with the accuracy and generalization capabilities of the model were evaluated on this part.

The model selection process involved a comparative evaluation of various pretrained models to identify those best suited for leaf disease detection. Pretrained architectures such DenseNet121, DenseNet201, EfficientNetB0, InceptionV3, MobileNet, MobileNetV2, NasNetMobile, ResNet50, ResNet152, VGG16, and VGG19 were prioritized due to their established effectiveness in image classification. Each model was assessed based on its performance in identifying disease features in leaf imagery, leveraging their deep layers and sophisticated feature extraction capabilities. Through comparison, these models demonstrated distinct strengths, with DenseNet169 and ResNet152 excelling in accuracy and feature depth, while MobileNetV2 offered an advantage in computational efficiency, making it suitable for real-time applications.

After choosing the models, we went ahead and adjusted their hyperparameters for best results. During this implementation step, we used our New Plant Disease dataset to train and assess the selected pre-trained models. Finding the most promising models that could precisely diagnose and categorize leaf illnesses and offer farmers a workable way to improve mango growing techniques was the aim.

Figure 11, which depicts our research strategy, shows how our project has developed from dataset selection to model selection and execution. It demonstrates our methodical and thorough approach, which guarantees the efficient application of deep learning algorithms for precise mango leaf disease identification.

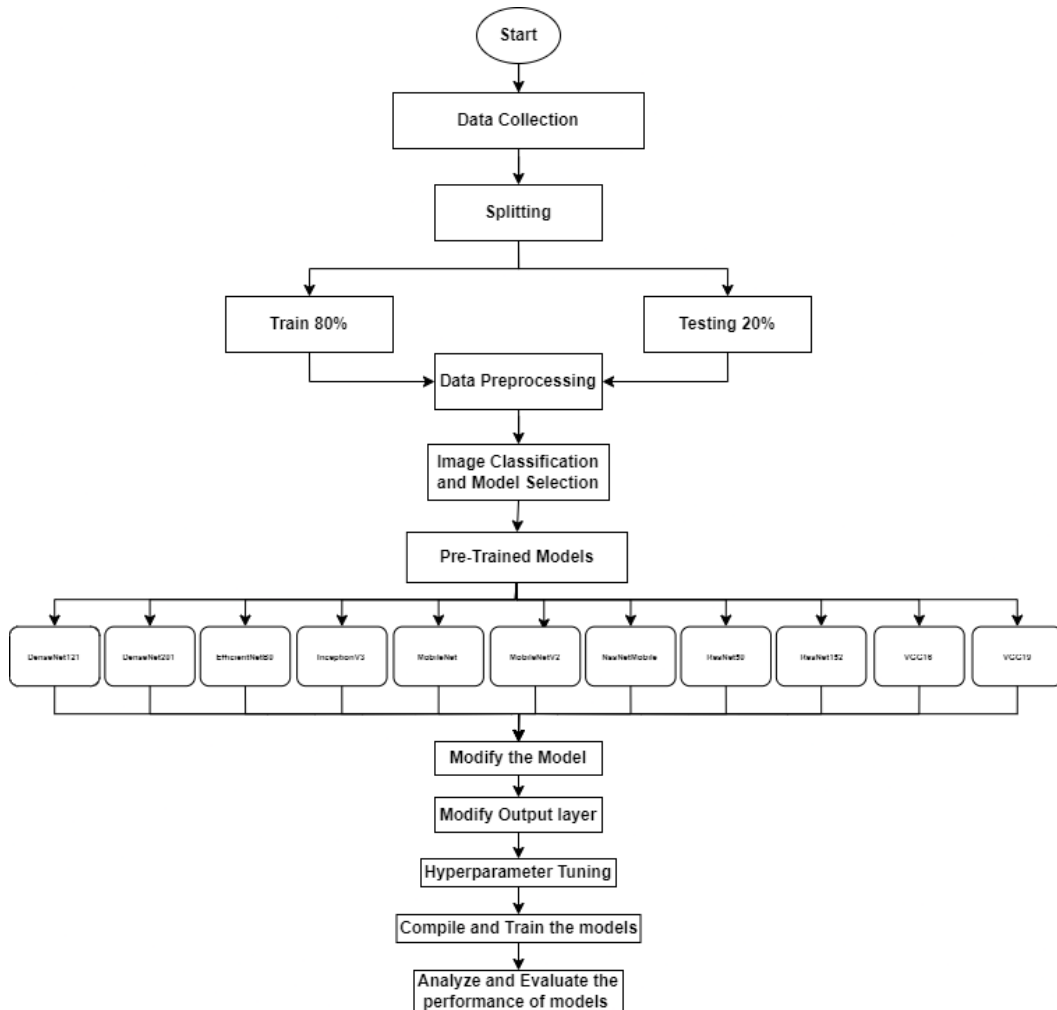


Figure 12. Workflow diagram for Implementation

4.2 Library Used

In this project, several essential libraries are used to handle data processing, visualization, and building deep learning models efficiently. Below is a summary of the libraries, including their names, required versions, and brief descriptions:

Table 2. Summary of listed libraries

Serial No.	Library	Version
1	Numpy	1.18.5+
2	Pandas	1.0.5+
3	Matplotlib	3.2.2+
4	OS	Built-in
5	TensorFlow	2.0+
6	Keras	2.4+
7	Seaborn	0.10.1+
8	Scikit-Learn	0.22

4.3 Structural View of Code

Table 3. Structural view of the code skeleton

Step 1	Import necessary libraries
Step 2	Load the data
Step 3	Prepare the data
Step 4	Visually represent sample data
Step 5	Create model
Step 6	Compile the model
Step 7	Train the model
Step 8	Evaluate the model and make predictions
Step 9	Calculate different performance matrix

4.4 Model Selection

In our research, we chose four pre-trained models for comparison and determine the best model above any else. These pre-trained models include VGG16, MobileNetV2, InceptionV3, and DenseNet169, each of which provides a distinct combination of performance and accuracy. We prioritized robustness and unique performance when selecting models. We carefully managed the transfer learning layer to ensure that the models can easily adapt to our specific measurement task, even if they have not previously shown similar data. So, careful selection and fine-tuning will improve the models' performance and efficiency in our workflow. Figure 13 depicts a diagram of model selection.

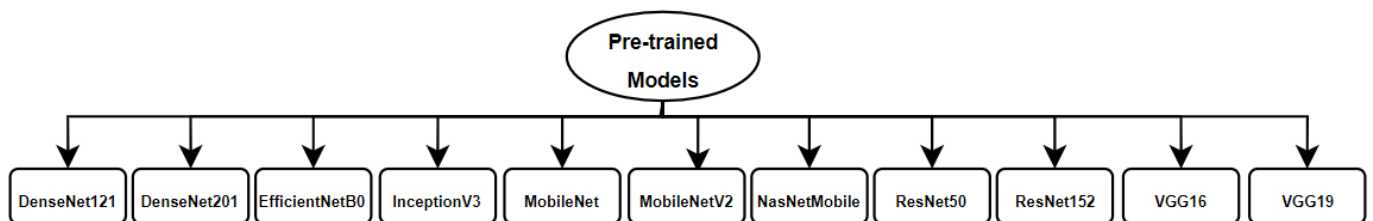


Figure 13. Diagram of model selection

4.5 Hyperparameter Tuning

Hyperparameters are the settings we provide when invoking a function or class. These settings are critical for optimizing an algorithm's performance. Accurate values can considerably improve algorithm efficiency and performance. Proper hyperparameter selection improves model efficiency and performance. Selecting optimal hyperparameter settings ensures the success of any machine learning method.

4.6 Design and Compile the Models

4.6.1 DenseNet121

DenseNet121 is a pre-trained model that trained on a huge dataset. First, we loaded the model and eliminated the original fully connected layers. Second, we updated it by adding a flatten layer. The final output layer is dense, having 38 neurons. We used the softmax activation function, which is suitable for multiclass classification. To freeze the top layer and base model, just the mode weights are retained. This procedure might be referred to as feature extraction. This setup allows for accurate picture feature extraction using the pre-trained DenseNet121 features.

We can then use the Adam optimizer to assemble the model. During compilation, we can apply categorical cross entropy, which is called the loss function and is appropriate for multi-class classification. Finally, we defined an accuracy metric to track the model's performance during training and evaluation.

4.6.2 DenseNet201

DenseNet201 is a pre-trained model that trained on a huge dataset. First, we loaded the model and eliminated the original fully connected layers. Second, we updated it by adding a flatten layer. The final output layer is dense, having 38 neurons. We used the softmax activation function, which is suitable for multiclass classification. To freeze the top layer and base model, just the mode weights are retained. This procedure might be referred to as feature extraction. This setup allows for accurate picture feature extraction using the pre-trained DenseNet201 features.

We can then use the Adam optimizer to assemble the model. During compilation, we can apply categorical cross entropy, which is called the loss function and is appropriate for multi-class classification. Finally, we defined an accuracy metric to track the model's performance during training and evaluation.

4.6.3 EfficientNetB0

EfficientNetB0 is a pre-trained model that trained on a huge dataset. First, we loaded the model and eliminated the original fully connected layers. Second, we updated it by adding a flatten layer. The final output layer is dense, having 38 neurons. We used the softmax activation function, which is suitable for multiclass classification. To freeze the top layer and base model, just the mode weights are retained. This procedure might be referred to as feature extraction. This setup allows for accurate picture feature extraction using the pre-trained EfficientNetB0 features.

We can then use the Adam optimizer to assemble the model. During compilation, we can apply categorical cross entropy, which is called the loss function and is appropriate for multi-class classification. Finally, we defined an accuracy metric to track the model's performance during training and evaluation.

4.6.4 InceptionV3

InceptionV3 is a pre-trained model that trained on a huge dataset. First, we loaded the model and eliminated the original fully connected layers. Second, we updated it by adding a flatten layer. The final output layer is dense, having 38 neurons. We used the softmax activation function, which is suitable for multiclass classification. To freeze the top layer and base model, just the mode weights are retained. This procedure might be referred to as feature extraction. This setup allows for accurate picture feature extraction using the pre-trained InceptionV3 features.

We can then use the Adam optimizer to assemble the model. During compilation, we can apply categorical cross entropy, which is called the loss function and is appropriate for multi-class classification. Finally, we defined an accuracy metric to track the model's performance during training and evaluation.

4.6.5 MobileNet

MobileNet is a pre-trained model that trained on a huge dataset. First, we loaded the model and eliminated the original fully connected layers. Second, we updated it by adding a flatten layer. The final output layer is dense, having 38 neurons. We used the softmax activation function, which is suitable for multiclass classification. To freeze the top layer and base model, just the mode weights are retained. This procedure might be referred to as feature extraction. This setup allows for accurate picture feature extraction using the pre-trained MobileNet features.

We can then use the Adam optimizer to assemble the model. During compilation, we can apply categorical cross entropy, which is called the loss function and is appropriate for multi-class classification. Finally, we defined an accuracy metric to track the model's performance during training and evaluation.

4.6.6 *MobileNetV2*

MobileNetV2 is a pre-trained model that trained on a huge dataset. First, we loaded the model and eliminated the original fully connected layers. Second, we updated it by adding a flatten layer. The final output layer is dense, having 38 neurons. We used the softmax activation function, which is suitable for multiclass classification. To freeze the top layer and base model, just the mode weights are retained. This procedure might be referred to as feature extraction. This setup allows for accurate picture feature extraction using the pre-trained MobileNetV2 features.

We can then use the Adam optimizer to assemble the model. During compilation, we can apply categorical cross entropy, which is called the loss function and is appropriate for multi-class classification. Finally, we defined an accuracy metric to track the model's performance during training and evaluation.

4.6.7 *NasNetMobile*

NasNetMobile is a pre-trained model that trained on a huge dataset. First, we loaded the model and eliminated the original fully connected layers. Second, we updated it by adding a flatten layer. The final output layer is dense, having 38 neurons. We used the softmax activation function, which is suitable for multiclass classification. To freeze the top layer and base model, just the mode weights are retained. This procedure might be referred to as feature extraction. This setup allows for accurate picture feature extraction using the pre-trained NasNetMobile features.

We can then use the Adam optimizer to assemble the model. During compilation, we can apply categorical cross entropy, which is called the loss function and is appropriate for multi-class classification. Finally, we defined an accuracy metric to track the model's performance during training and evaluation.

4.6.8 *ResNet50*

ResNet50 is a pre-trained model that trained on a huge dataset. First, we loaded the model and eliminated the original fully connected layers. Second, we updated it by adding a flatten layer. The final output layer is dense, having 38 neurons. We used the softmax activation function, which is suitable for multiclass classification. To freeze the top layer and base model, just the mode weights are retained. This procedure might be referred to as feature extraction. This setup allows for accurate picture feature extraction using the pre-trained ResNet50 features.

We can then use the Adam optimizer to assemble the model. During compilation, we can apply categorical cross entropy, which is called the loss function and is appropriate for multi-class classification. Finally, we defined an accuracy metric to track the model's performance during training and evaluation.

4.6.9 *ResNet152*

ResNet152 is a pre-trained model that trained on a huge dataset. First, we loaded the model and eliminated the original fully connected layers. Second, we updated it by adding a flatten layer. The final output layer is dense, having 38 neurons. We used the softmax activation function, which is suitable for multiclass classification. To freeze the top layer and base model, just the mode weights are retained. This procedure might be referred to as feature extraction. This setup allows for accurate picture feature extraction using the pre-trained ResNet152 features.

We can then use the Adam optimizer to assemble the model. During compilation, we can apply categorical cross entropy, which is called the loss function and is appropriate for multi-class classification. Finally, we defined an accuracy metric to track the model's performance during training and evaluation.

4.6.10 *VGG16*

VGG16 is a pre-trained model that trained on a huge dataset. First, we loaded the model and eliminated the original fully connected layers. Second, we updated it by adding a flatten layer. The final output layer is dense, having 38 neurons. We used the softmax activation function, which is suitable for multiclass classification. To freeze the top layer and base model, just the mode weights are retained. This procedure might be referred to as feature extraction. This setup allows for accurate picture feature extraction using the pre-trained VGG16 features.

We can then use the Adam optimizer to assemble the model. During compilation, we can apply categorical cross entropy, which is called the loss function and is appropriate for multi-class classification. Finally, we defined an accuracy metric to track the model's performance during training and evaluation.

4.6.11 VGG19

VGG19 is a pre-trained model that trained on a huge dataset. First, we loaded the model and eliminated the original fully connected layers. Second, we updated it by adding a flatten layer. The final output layer is dense, having 38 neurons. We used the softmax activation function, which is suitable for multiclass classification. To freeze the top layer and base model, just the model weights are retained. This procedure might be referred to as feature extraction. This setup allows for accurate picture feature extraction using the pre-trained VGG19 features.

We can then use the Adam optimizer to assemble the model. During compilation, we can apply categorical cross entropy, which is called the loss function and is appropriate for multi-class classification. Finally, we defined an accuracy metric to track the model's performance during training and evaluation.

5. Result Analysis

5.1 Training and Analysis of Models

5.1.1 DenseNet121

We train this model by giving both the training and testing datasets. The steps per epoch are equal to the number of training samples divided by the batch size (128). During each epoch, the training process will iterate 1/128 of the total number of samples in the training dataset. The training will run for 50 epochs because we set epochs to 50. We also use early stopping and callbacks. This will monitor and store the model with the highest validation accuracy. We also save the training history in a variable and display it on a graph. This model has a test accuracy of 0.9217 and validation accuracy of 0.9441. Figures 14 and 15 show graphs comparing training accuracy with validation accuracy, as well as training loss vs validation loss.

5.1.2 DenseNet201

We train this model by giving both the training and testing datasets. The steps per epoch are equal to the number of training samples divided by the batch size (128). During each epoch, the training process will iterate 1/128 of the total number of samples in the training dataset. The training will run for 50 epochs because we set epochs to 50. We also use callbacks. This will monitor and store the model with the highest validation accuracy. We also save the training history in a variable and display it on a graph. This model has a test accuracy of 0.9252 and validation accuracy of 0.9391. Figures 14 and 15 show graphs comparing training accuracy with validation accuracy, as well as training loss vs validation loss.

5.1.3 EfficientNetB0

We train this model by giving both the training and testing datasets. The steps per epoch are equal to the number of training samples divided by the batch size (64). During each epoch, the training process will iterate 1/64 of the total number of samples in the training dataset. The training will run for 50 epochs because we set epochs to 50. We also use callbacks. This will monitor and store the model with the highest validation accuracy. We also save the training history in a variable and display it on a graph. This model has a test accuracy of 0.8320 and validation accuracy of 0.8779. Figures 14 and 15 show graphs comparing training accuracy with validation accuracy, as well as training loss vs validation loss.

5.1.4 InceptionV3

We train this model by giving both the training and testing datasets. The steps per epoch are equal to the number of training samples divided by the batch size (64). During each epoch, the training process will iterate 1/64 of the total number of samples in the training dataset. The training will run for 50 epochs because we set epochs to 50. We also use callbacks. This will monitor and store the model with the highest validation accuracy. We also save the training history in a variable and display it on a graph. This model has a test accuracy of 0.8044 and validation accuracy of 0.8276. Figures 14 and 15 show graphs comparing training accuracy with validation accuracy, as well as training loss vs validation loss.

5.1.5 MobileNet

We train this model by giving both the training and testing datasets. The steps per epoch are equal to the number of training samples divided by the batch size (64). During each epoch, the training process will iterate 1/64 of the total number of samples in

the training dataset. The training will run for 50 epochs because we set epochs to 50. We also use callbacks. This will monitor and store the model with the highest validation accuracy. We also save the training history in a variable and display it on a graph. This model has a test accuracy of 0.8875 and validation accuracy of 0.9045. Figures 14 and 15 show graphs comparing training accuracy with validation accuracy, as well as training loss vs validation loss.

5.1.6 *MobileNetV2*

We train this model by giving both the training and testing datasets. The steps per epoch are equal to the number of training samples divided by the batch size (64). During each epoch, the training process will iterate 1/64 of the total number of samples in the training dataset. The training will run for 50 epochs because we set epochs to 50. We also use callbacks. This will monitor and store the model with the highest validation accuracy. We also save the training history in a variable and display it on a graph. This model has a test accuracy of 0.8513 and validation accuracy of 0.8729. Figures 14 and 15 show graphs comparing training accuracy with validation accuracy, as well as training loss vs validation loss.

5.1.7 *NasNetMobile*

We train this model by giving both the training and testing datasets. The steps per epoch are equal to the number of training samples divided by the batch size (64). During each epoch, the training process will iterate 1/64 of the total number of samples in the training dataset. The training will run for 50 epochs because we set epochs to 50. We also use callbacks. This will monitor and store the model with the highest validation accuracy. We also save the training history in a variable and display it on a graph. This model has a test accuracy of 0.7944 and validation accuracy of 0.7783. Figures 14 and 15 show graphs comparing training accuracy with validation accuracy, as well as training loss vs validation loss.

5.1.8 *ResNet50*

We train this model by giving both the training and testing datasets. The steps per epoch are equal to the number of training samples divided by the batch size (64). During each epoch, the training process will iterate 1/64 of the total number of samples in the training dataset. The training will run for 50 epochs because we set epochs to 50. We also use callbacks. This will monitor and store the model with the highest validation accuracy. We also save the training history in a variable and display it on a graph. This model has a test accuracy of 0.8798 and validation accuracy of 0.9133. Figures 14 and 15 show graphs comparing training accuracy with validation accuracy, as well as training loss vs validation loss.

5.1.9 *ResNet152*

We train this model by giving both the training and testing datasets. The steps per epoch are equal to the number of training samples divided by the batch size (64). During each epoch, the training process will iterate 1/64 of the total number of samples in the training dataset. The training will run for 50 epochs because we set epochs to 50. We also use callbacks. This will monitor and store the model with the highest validation accuracy. We also save the training history in a variable and display it on a graph. This model has a test accuracy of 0.9011 and validation accuracy of 0.9272. Figures 14 and 15 show graphs comparing training accuracy with validation accuracy, as well as training loss vs validation loss.

5.1.10 *VGG16*

We train this model by giving both the training and testing datasets. The steps per epoch are equal to the number of training samples divided by the batch size (64). During each epoch, the training process will iterate 1/64 of the total number of samples in the training dataset. The training will run for 50 epochs because we set epochs to 50. We also use callbacks. This will monitor and store the model with the highest validation accuracy. We also save the training history in a variable and display it on a graph. This model has a test accuracy of 0.8602 and validation accuracy of 0.8853. Figures 14 and 15 show graphs comparing training accuracy with validation accuracy, as well as training loss vs validation loss.

5.1.11 *VGG19*

We train this model by giving both the training and testing datasets. The steps per epoch are equal to the number of training samples divided by the batch size (128). During each epoch, the training process will iterate 1/128 of the total number of samples in the training dataset. The training will run for 50 epochs because we set epochs to 50. We also use callbacks. This will monitor and store the model with the highest validation accuracy. We also save the training history in a variable and display it on a graph. This model has a test accuracy of 0.8803 and validation accuracy of 0.9011. Figures 14 and 15 show graphs comparing training accuracy with validation accuracy, as well as training loss vs validation loss.

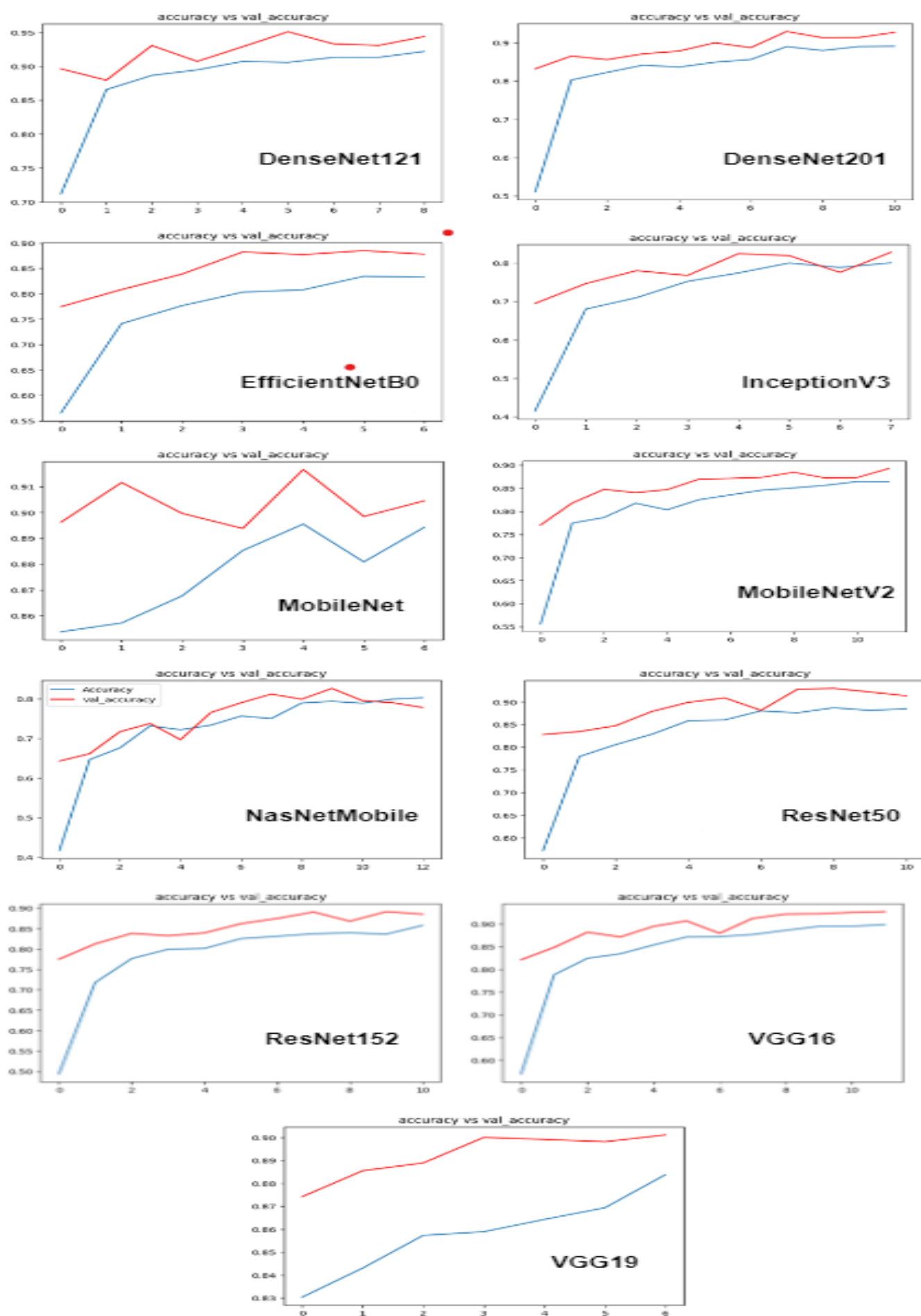


Figure 14. Testing_accuracy Vs Validation_accuracy

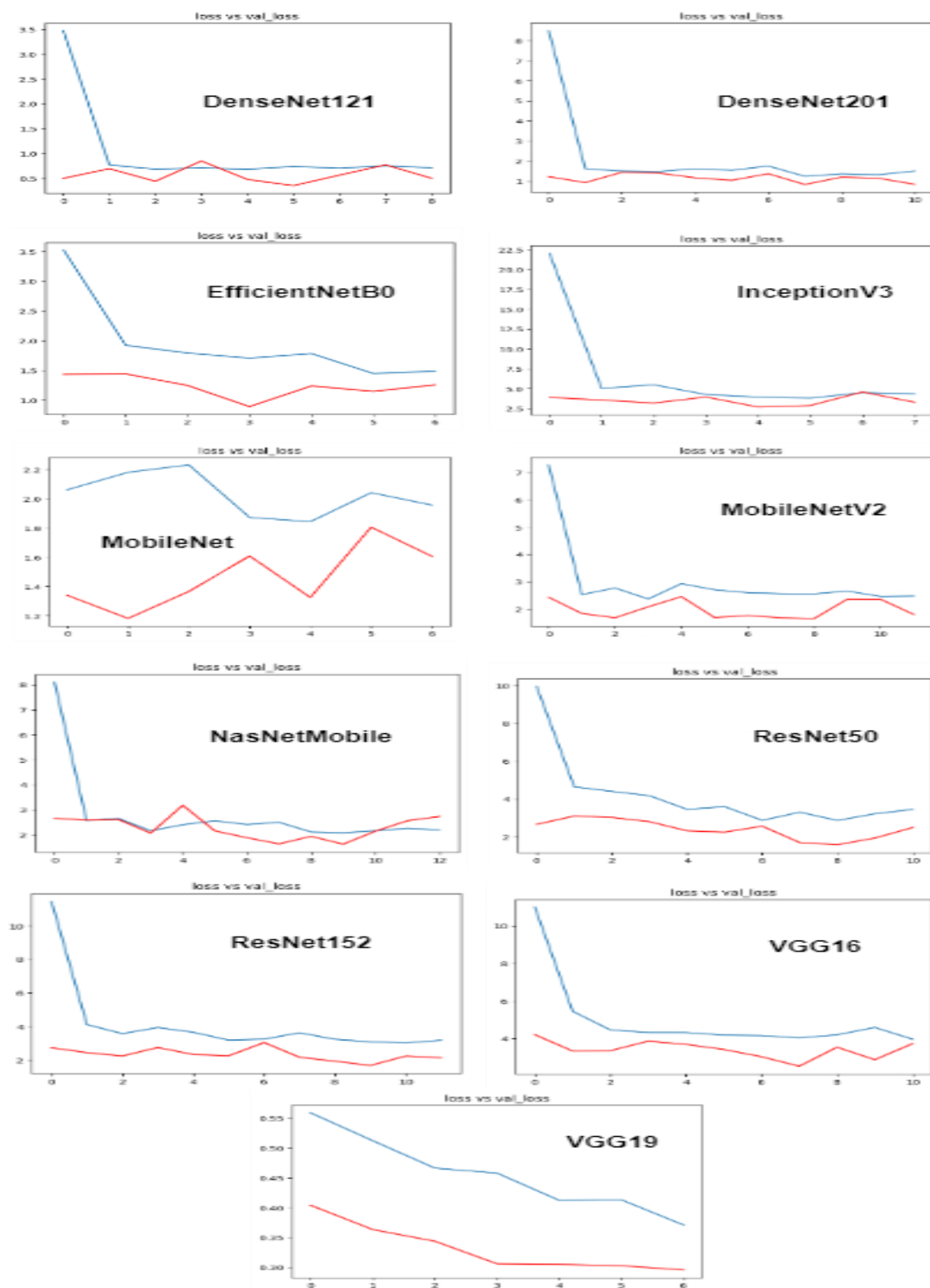


Figure 15. Training_loss Vs Validation_loss

5.2 Analysis

Table 4. Study Summary

Model	Optimizer	Batch Size	Test Accuracy	Validation Accuracy	Test loss	Validation loss
DenseNet121	Adam	128	0.9217	0.9441	0.7046	0.5057
DenseNet201	Adam	128	0.9252	0.9391	0.7064	0.6584
EfficientNetB0	Adam	64	0.8320	0.8779	1.4255	1.2561
InceptionV3	Adam	64	0.8044	0.8276	4.2939	3.2912
MobileNet	Adam	64	0.8872	0.9045	2.0344	1.6059
MobileNetV2	Adam	64	0.8513	0.8729	2.4559	2.2513
NasNetMobile	Adam	64	0.7944	0.7783	2.2169	2.7284
ResNet50	Adam	64	0.8798	0.9133	3.7901	2.4866
ResNet152	Adam	64	0.9011	0.9272	3.1464	2.1385
VGG16	Adam	64	0.8602	0.8853	3.8563	3.7241
VGG19	Adam	128	0.8803	0.9100	0.3718	0.2956

Table 5. Size Comparison

Model	Size (MB)
DenseNet121	50.0
DenseNet201	113
EfficientNetB0	43.5
InceptionV3	141
MobileNet	34.4
MobileNetV2	36.4
NasNetMobile	41.3
ResNet50	134
ResNet152	268
VGG16	67.1
VGG19	87.3

6. Conclusion

The performance and model size analysis reveal trade-offs between accuracy, computational efficiency, and storage requirements, essential considerations for selecting an optimal model for multiclass classification tasks.

DenseNet121 and DenseNet201 stand out for their high accuracy (92.17% and 92.52% respectively) and relatively low test and validation losses, making them reliable for robust classification. DenseNet121 is favorable, as it offers high accuracy with a compact size of 50 MB, compared to DenseNet201's larger size of 113 MB. This balance makes DenseNet121 an efficient choice for applications that require both accuracy and storage efficiency.

EfficientNetB0 provides moderate accuracy (83.20%) with a compact size of 43.5 MB, suitable for environments where model size is critical. However, its relatively higher losses indicate that further tuning may be needed for optimal performance.

MobileNet and MobileNetV2 are among the smallest models (34.4 MB and 36.4 MB, respectively), making them viable for lightweight applications. MobileNet's higher accuracy (88.72%) and moderate losses compared to MobileNetV2 make it a preferable choice when storage constraints are a priority, but a reasonable level of accuracy is still desired.

ResNet50 and ResNet152 performed well in terms of accuracy, particularly ResNet152, which achieved 90.11% test accuracy. However, their sizes (134 MB and 268 MB, respectively) make them more suitable for resource-rich environments where model performance is prioritized over compactness.

VGG16 and VGG19 achieved stable accuracy (around 86-88%) but are larger than many other models of comparable accuracy, at 67.1 MB and 87.3 MB. Although VGG19 performs slightly better than VGG16, its higher storage requirements make it less suitable for storage-constrained deployments.

In summary, DenseNet121 is the best all-around choice, combining high accuracy, low loss, and moderate model size, making it suitable for applications that balance accuracy with storage constraints. For extremely storage-sensitive applications, MobileNet

is a practical alternative with smaller size and decent accuracy, while ResNet152 is ideal for applications prioritizing deep feature extraction with high accuracy and minimal concern for model size.

7. Future Works

For future work, several areas could further enhance the model performance, efficiency, and applicability in multiclass classification tasks involving image data:

- **Exploring Other Architectures:** New architectures, such as more recent versions of EfficientNet or Vision Transformers (ViTs), could be explored to balance accuracy and computational efficiency further. These models may offer better generalization while being optimized for lower resource consumption.
- **Model Optimization for Deployment:** Techniques such as quantization, pruning, or knowledge distillation could be applied to reduce model size and inference time without sacrificing accuracy. This approach would be particularly beneficial for deploying high-performing models (like ResNet152 and DenseNet201) in real-time or resource-constrained environments.
- **Dataset Expansion:** Adding more photographs from a different location and environmental circumstances into the dataset will further improve the robustness of the models. More classification of illnesses and stages of infection will be able to increase the ability to diagnose.
- **Ensemble Learning:** Combining multiple models in an ensemble could improve classification accuracy and robustness, especially for classes that individual models may struggle to classify. This approach may offer a balance between accuracy and model reliability.
- **IoT Integration:** By integrating the models with Internet of Things (IoT) devices, continuous monitoring of crops is made possible, giving farmers access to real-time data collection and instant response.

References

- [1] L. W. S. J. P. P. E. N. M. A. N. Serge Savary, "The global burden of pathogens and pests on major food crops," *Nat. Ecol. Evol.*, pp. 430-439, 2019.
- [2] V. K. H. D. J. Gittaly Dhingra, "Study of digital image processing techniques for leaf disease detection and classification," *Multimed Tools Appl*, vol. 77, p. 19951–20000, 2018.
- [3] S. F. J. T. P.-L. S.-C. Justine Boulent, "Convolutional Neural Networks for the Automatic Identification of Plant Diseases," *Front. Plant Sci*, 2019.
- [4] M. &. K. A. &. M. A. Brahim, "Deep Learning for Tomato Diseases: Classification and Symptoms Visualization.," *Applied Artificial Intelligence*, vol. 31, pp. 1-17, 2017.
- [5] A. Aissaoui, I. Aloui, S. Boudibi, Z.-E. Khomri and A. Foughalia, "Deep Learning Approaches for Plant Diseases Identification and Classification: A Comprehensive Review," *8th International Conference on Image and Signal Processing and their Applications (ISPA)*, Biskra, Algeria, 2024, pp. 1-11, 2024.
- [6] K. K. K. A. Y. B. S. V. P. Ramesh Kumar Mojjada, "WITHDRAWN: Detection of plant leaf disease using digital image processing," *Materials Today: Proceedings*, 2020.
- [7] K. K. B. K. Vibhor Kumar Vishnoi, "Plant disease detection using computational intelligence and image processing," *J Plant Dis Prot*, vol. 128, pp. 19-53, 2021.
- [8] M. &. G. K. Applalanaidu, "A Review of Machine Learning Approaches in Plant Leaf Disease Detection and Classification," in *Third International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV)*, 2021.
- [9] K. E. S. M. K. S. P. Prathusha, "Plant Disease Detection Using Machine Learning Algorithms," in *International Conference on Computational and Bio Engineering*, Berlin/Heidelberg, Germany, 2019.
- [10] R. P. Shradha S. Pradhan, "Comparison of Deep Learning Approaches for Plant Disease Detection," in *Proceedings of International Conference on Wireless Communication*, Singapore, 2020.
- [11] P. S. P. K. Guneet Sachdeva, "Plant leaf disease classification using deep Convolutional neural network with Bayesian learning," *Mater. Today Proc.*, vol. 45, pp. 5584-5590, 2021.
- [12] M. S. M. K. E. S. Houda Orchi, "Automation of Crop Disease Detection through Conventional Machine Learning and Deep Transfer Learning Approaches," *Agriculture*, vol. 13, p. 352, 2023.
- [13] D. P. H. M. S. Sharada P. Mohanty, "Using Deep Learning for Image-Based Plant Disease Detection," *Front. Plant Sci*, 2016.
- [14] R. J. P. M. M. A. N. S. Iniyar, "Plant Disease Identification and Detection Using Support Vector Machines and Artificial Neural Networks," in *Advances in Intelligent Systems and Computing*, Singapore, Springer, 2020.
- [15] R. N. S. S. A. S. V. V. S. Arivazhagan, "Detection of unhealthy region of plant leaves and classification of plant leaf diseases using texture features," *Agricultural Engineering International: CIGR*, vol. 15, pp. 211-217, 2013.
- [16] M. S. M. K. Houda Orchi, "A General Survey on Plants Disease Detection Using Image Processing, Deep Transfer Learning and Machine Learning Techniques," in *UNet 2021*, 2021.

- [17] A. P. U. I. A. M. M. G. S.-E. A. B. A. A.-G. David Argüeso, "Few-Shot Learning approach for plant disease classification using images taken in the field," *Computers and Electronics in Agriculture*, vol. 175, 2020.
- [18] D. M. A. T. X.E. Pantazi, "Automated leaf disease detection in different crop species through image features analysis and One Class Classifiers," *Computers and Electronics in Agriculture*, vol. 156, pp. 96-104, 2019.
- [19] U. A. P. S. Jatin Arora, "Classification of Maize leaf diseases from healthy leaves using Deep Forest," *Journal of Artificial Intelligence and Systems*, vol. 2, pp. 14-26, 2020.
- [20] J. Y. Z. L. F. Q. Zhe Tang, "Grape disease image classification based on lightweight convolution neural networks and channelwise attention," *Computers and Electronics in Agriculture*, vol. 178, 2020.
- [21] A. C. a. A. P. Anshul Bhatia, "Application of extreme learning machine in plant disease prediction for highly imbalanced dataset," *Journal of Statistics and Management Systems*, vol. 23(6), p. 1059-1068, 2020.
- [22] K. W. B. L. O. H. W. P. K. N. V. Chawla, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence*, vol. 16, pp. 321-357, 2002.
- [23] U. K. Y. A. J. M. N. A. K. Haziqah Shamsudin, "Combining oversampling and undersampling techniques for imbalanced classification: A comparative study using credit card fraudulent transaction dataset," in *IEEE 16th International Conference on Control & Automation (ICCA)*, Singapore, 2020.
- [24] A. S. S. A. A. S. S. G. Mohit Agarwal, "ToLeD: Tomato Leaf Disease Detection using Convolution Neural Network," *Procedia Computer Science*, vol. 167, pp. 293-301, 2020.
- [25] S. N. N. Y. S. P. Rutu Gandhi, "Plant disease detection using CNNs and GANs as an augmentative approach," in *IEEE International Conference on Innovative Research and Development (ICIRD)*, Bangkok, Thailand, 2018.
- [26] M. T. A. Y. R. E. Faruk Ahmed, "Machine Learning-Based Tea Leaf Disease Detection: A Comprehensive Review," 2023.
- [27] H. X. Z. G. C. A. W. Y. L. L. H. Y. Tang Z, "A Precise Image-Based Tomato Leaf Disease Detection Approach Using PLPNet," *Plant Phenomics*, 2023.
- [28] V. K. a. S. V. S. Mehta, "Advancing Agricultural Practices: Federated Learning-based CNN for Mango Leaf Disease Detection," in *3rd International Conference on Intelligent Technologies (CONIT)*, Hubli, India, 2023.
- [29] google, "colab," [Online]. Available: <https://colab.research.google.com/notebooks>. [Accessed 12 September 2024].
- [30] S. Bhattarai, "kaggle," [Online]. Available: <https://www.kaggle.com/datasets/vipooooo/new-plant-diseases-dataset/data>. [Accessed 12 September 2024].
- [31] S. Mohanty, "github," [Online]. Available: <https://github.com/spMohanty/PlantVillage-Dataset>. [Accessed 12 September 2024].
- [32] Z. L. L. v. d. M. K. Q. W. Gao Huang, "Densely Connected Convolutional Networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [33] S. Das, "Medium," [Online]. Available: https://miro.medium.com/v2/resize:fit:828/format:webp/1*4kPpyvHv73ypzTvo6y8J9w.png. [Accessed 1 November 2024].
- [34] Q. V. L. Mingxing Tan, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *International Conference on Machine Learning*, 2019.
- [35] V. Agarwal, "Medium," [Online]. Available: https://miro.medium.com/v2/resize:fit:2000/format:webp/1*rnhgFRXetwD8PvxhZlpwIA.png. [Accessed 1 November 2024].
- [36] K. S. L. Tonny Binoy, "Comparative analysis of vehicle make and model recognition using deep learning techniques," *2nd International Conference on Intelligent Computing, Instrumentation and Control Technologies (ICICT)*, IEEE, vol. 1, pp. 1298-1305, 2019.
- [37] V. V. S. I. J. S. Z. W. Christian Szegedy, "Rethinking the Inception Architecture for Computer Vision," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2818-2826, 2016.
- [38] A. Brital, "Medium," [Online]. Available: https://miro.medium.com/v2/resize:fit:1400/format:webp/1*uVRWg5TG5_3W18qdhp03XQ.png. [Accessed 1 November 2024].
- [39] M. Z. B. C. D. K. W. W. T. W. M. A. H. A. Andrew G. Howard, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," 2017. [Online]. Available: <https://arxiv.org/abs/1704.04861>. [Accessed 1 November 2024].
- [40] K. & D. R. Saini, "A systematic scoping review of the analysis of COVID-19 disease using chest X-ray images with deep learning models," *Journal of Autonomous Intelligence*, 2023.
- [41] Q. V. L. Barret Zoph, "Neural Architecture Search with Reinforcement Learning," Google Brain, 2018.
- [42] V. V. J. S. Q. V. L. Barret Zoph, "Learning Transferable Architectures for Scalable Image Recognition," Google Brain, 2018.
- [43] A. Z. Karen Simonyan, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in *ICLR*, 2015.